

LAPORAN RESMI PRAKTIKUM

DevSecOps — Bab 1

Fondasi Teoretis dan Kerangka Kerja DevSecOps



Nama : Fawwaz Mufid Wardaya
Kelas : D4 Teknik Informatika B
NRP : 3126640050

**PROGRAM STUDI TEKNIK INFORMATIKA
DEPARTEMEN TEKNIK INFORMATIKA DAN KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2026**

Pendahuluan

Latar Belakang

DevSecOps adalah pendekatan yang mengintegrasikan keamanan ke dalam keputusan, alur kerja, otomasi, dan tanggung jawab sepanjang siklus hidup perangkat lunak — bukan sekadar penambahan scanner ke pipeline. DevOps memperpendek jarak antara kode dan umpan balik produksi, tetapi kecepatannya memperbesar konsekuensi kesalahan; karena itu kontrol keamanan ditempatkan pada jalur yang sama dengan delivery sebagai guardrail yang terversi dan dapat diaudit. Pada Bab 1 ini disusun baseline laboratorium (direktori kerja + rekaman versi komponen) sebagai acuan reproducible untuk seluruh eksperimen berikutnya.

Rumusan Masalah

Mengapa DevSecOps tidak dapat direduksi menjadi penambahan scanner pada pipeline?

Evidence apa yang membedakan klaim kontrol dari kontrol yang terverifikasi?

Bagaimana shared responsibility memengaruhi ownership risiko dan tindak lanjut temuan?

Bagaimana menyusun baseline laboratorium yang aman, legal, dan dapat direplikasi?

Pembahasan

Dasar Teori

DevSecOps membawa kontrol ke dalam lifecycle: threat modeling saat desain, secure coding & secret scanning saat code, SAST/SCA saat build, DAST & runtime detection saat operasi. Shift-left memberi umpan balik sedini mungkin; shift-right melengkapinya lewat pengujian runtime. Security gate adalah keputusan kebijakan berbasis hasil pemeriksaan (input terdefinisi, threshold transparan, waiver dengan pemilik & kedaluwarsa). Evidence as product berarti keluaran keamanan (SARIF, JUnit XML, SBOM, signature, policy evaluation, log) harus traceable ke commit/digest/waktu/tool version — screenshot saja tidak cukup sebagai sumber kebenaran.

Pemetaan NIST SSDF:

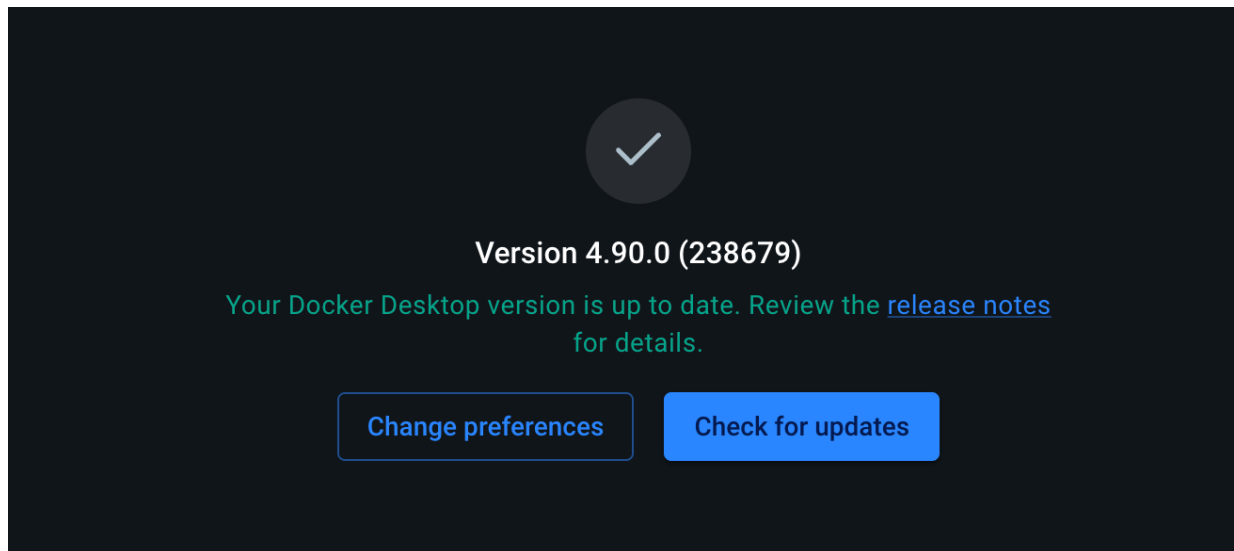
Kelompok	Makna	Implementasi
PO	Prepare the Organization	Peran, baseline, gate, threat model, metrik
PS	Protect the Software	Proteksi source, secret, build, registry, signature
PW	Produce Well-Secured Software	Secure coding, SAST, SCA, SBOM, DAST, hardening
RV	Respond to Vulnerabilities	Triage, remediation, runtime detection, incident

Praktikum 1 — Baseline Laboratorium

Perintah:

```
mkdir -p ~/devsecops-lab/{app,policy,reports,sbom,keys} && cd ~/devsecops-lab
docker version && docker compose version && git --version
openssl version && curl --version
docker info --format '{{json .SecurityOptions}}'
```

Hasil versi (ringkas): Docker 29.7.2 (client darwin/arm64; server Engine 29.7.2 linux/arm64, containerd v2.3.3, runc 1.4.3), Compose v5.5.0, Git 2.50.1, OpenSSL 3.6.3, curl 8.7.1.



Gambar: docker version menampilkan Client dan Server Docker Desktop 4.89.0

```
└─ docker info --format '{{json .SecurityOptions}}'
["name=seccomp,profile=builtin","name=cgroupns"]
```

Gambar: docker info SecurityOptions = [seccomp profile=builtin, cgroupns]

Analisis: SecurityOptions tercatat [seccomp profile=builtin, cgroupns] — dua mekanisme host yang tersedia pada LinuxKit Docker Desktop. Tidak ada AppArmor/SELinux maupun rootless, sehingga hardening container pada bab berikutnya wajib dilakukan eksplisit via Dockerfile/Compose (non-root, read-only fs, drop capabilities). Hasil ini sesuai pelajaran bab: SecurityOptions menunjukkan mekanisme host yang tersedia, bukan bukti container telah hardened.

Evaluasi dan Latihan Mandiri

1. Mengapa DevSecOps tidak dapat direduksi menjadi penambahan scanner?

Scanner hanya menghasilkan temuan; pipeline tetap tidak aman bila tidak ditentukan apa yang membuat build gagal, siapa pemilik waiver, dan evidence apa yang disimpan. DevSecOps adalah sistem sosio-teknis yang menempatkan keamanan sebagai kriteria pada setiap tahap (desain→code→build→test→release→deploy→operate), dengan pembagian peran (PO/dev/platform/security/ops/auditor) yang berbagi akuntabilitas, bukan duplikasi tugas. Scanner tanpa budaya, gate, dan guardrail hanyalah ketidakjelasan yang diotomasi.

2. Evidence apa membedakan klaim kontrol dari kontrol terverifikasi?

Evidence yang traceable & machine-readable: SARIF, JUnit XML, SBOM CycloneDX/SPDX, attestations/signature, hasil policy evaluation, log deploy, event runtime — semuanya mengikat commit, image digest, waktu, tool version, dan keputusan waiver. Screenshot tidak cukup karena sulit diproses ulang dan kehilangan konteks. Klaim tanpa evidence demikian hanya pernyataan; kontrol terverifikasi memiliki bukti yang dapat diaudit dan diuji ulang.

3. Bagaimana shared responsibility memengaruhi ownership risiko?

Akuntabilitas didistribusikan tanpa menduplikasi tugas: PO tetapkan toleransi risiko, dev perbaiki temuan, platform sediakan build aman, security buat threat model/rule, ops kelola hardening/deteksi, auditor nilai evidence. Setiap temuan memiliki risk owner, threshold remediasi, dan jalur eskalasi (RACI, security champion, review dua orang). Waiver, bila diberikan, bersifat terdokumentasi, terbatas waktu, dan berpemilik — bukan ad hoc.

Penutup

Kesimpulan

DevSecOps adalah mekanisme pengelolaan risiko dan evidence pada sistem delivery, bukan penambahan scanner. Baseline laboratorium Bab 1 berhasil disusun: direktori ~/devsecops-lab dengan struktur app/policy/reports/sbom/keys, versi klien & server direkam (Docker 29.7.2, containerd v2.3.3, runc 1.4.3, Compose v5.5.0, Git 2.50.1, OpenSSL 3.6.3, curl 8.7.1), dan SecurityOptions tercatat [seccomp, cgroupns]. Karena AppArmor/SELinux/rootless tidak tersedia, hardening eksplisit wajib pada bab berikutnya. Baseline ini menjadi acuan reproducible untuk seluruh eksperimen.

Daftar Pustaka

Materi Ajar DevOps (DevSecOps Terintegrasi) — Bab 1. Politeknik Elektronika Negeri Surabaya, 2026.

NIST. (2022). Secure Software Development Framework (SSDF) v1.1 — SP 800-218.

OWASP. DevSecOps Guideline. <https://owasp.org/www-project-devsecops-guideline/>

Docker Inc. (2026). Docker Documentation — Engine, Security Options. <https://docs.docker.com/>

Akses terakhir: 8 September 2026.